

AGENT RELIABILITY ENGINEERING

The Manifesto

Agents are intelligent.
They are not yet reliable.

Without reliability, you only have liability.

Version 1.0 · July 2026 — open for contribution
Proposed by Mike Hogan

86% of enterprises have deployed agents. Only 34% trust them¹
40% are expected to demote or decommission autonomous agents²
Reliability, not intelligence, is now the adoption bottleneck

The next decade will not be won
by the smartest agents.
It will be won by the most reliable ones.

¹ [Boomi / Forrester \(2026\)](#): 86% of enterprises have moved beyond pilots, yet only 34% trust the actions their agents take; organizations in “agentic chaos” are exposed to average additional costs of \$2.1M from failures, fines, and rework.

² [Gartner](#) (May 2026): by 2027, 40% of enterprises are predicted to demote or decommission autonomous AI agents due to governance failures.

The Inflection

Every major shift in computing has required a new engineering discipline.

Mainframes created operations. Distributed systems demanded a new discipline; Google formalized it as Site Reliability Engineering. Cloud created Platform Engineering. Containers created a new operational model around Kubernetes.

Now software has begun to perform tasks. Agents no longer simply answer questions. They make decisions, call APIs, write code, move money, coordinate systems, and operate inside regulated environments with very real consequences. And their velocity demands automated compliance and governance.

“Some of these agents will live only milliseconds. You cannot hand-write policy for a system that is born, acts, and disappears faster than a human can read a ticket.”

We have spent years making agents more intelligent. We have spent comparatively little time making them reliable. That gap is becoming the defining engineering challenge of the next decade.

Enterprise evidence already shows the cost. Roughly half of enterprises that shipped agents based on internal evaluations later experienced customer-facing failures.³ Two-thirds of organizations that have deployed agents do not trust the actions those agents take.⁴ Adoption has raced ahead of reliability, exposing companies to liability.

Intelligence Is No Longer Enough

For the past several years, the industry has optimized almost exclusively for intelligence: larger models, better benchmarks, more capable reasoning, more tools, and more autonomy. That strategy worked while AI primarily answered questions. It breaks down once software begins taking action. An intelligent agent that cannot be trusted becomes an operational liability. The next generation of engineering must optimize for a different objective: reliable autonomous behavior.

³ [VentureBeat](#) / VB Pulse survey, June 2026, 157 enterprise respondents (2026): roughly half of enterprises that shipped agents or LLM features based on internal evaluations later experienced customer-facing failures; only a small minority fully trust automated evaluation.

⁴ Boomi / Forrester (2026): 86% of enterprises have moved beyond pilots, yet only 34% trust the actions their agents take; organizations in “agentic chaos” report average additional costs of \$2.1M from failures, fines, and rework.

Related Work

The term “Agent Reliability Engineering” and the core claim that reliability, not intelligence, is the primary bottleneck already appear in scattered places: Karpathy’s “march of nines”⁵ framing of production reliability, Gartner’s prediction that >40% of agentic AI projects will be canceled by end of 2027⁶, practitioner blogs and role definitions, academic efforts such as “Towards a Science of AI Agent Reliability”⁷, and commercial offerings such as Galileo’s Agent Reliability Platform⁸.

None of these, however, formalize ARE as a coherent engineering discipline with explicit principles, practices, and a call to build in the open. This document is the first manifesto that attempts to do so.

Reliability Surface

We define the **reliability surface** of an AI agent as its multi-dimensional behavioral profile under realistic conditions, independent of raw task success rate. Following Rabanser et al., it comprises four dimensions and twelve metrics:

- **Consistency:** outcome, trajectory (distributional + sequential), and resource consistency
- **Robustness:** fault tolerance, environmental robustness, and prompt/input perturbation robustness
- **Predictability:** calibration, discrimination, and Brier score
- **Safety:** policy compliance and harm severity

Leaders should fund visibility into this surface, not just accuracy, *before* production deployment.

The Smartest Intern

Imagine hiring an exceptionally smart intern. They understand everything immediately. They write elegant code. They surprise the team with insight.

But they only finish about half the work they start. Their consistency declines as projects grow more complex. The larger the team, the more their performance drifts. And when something goes wrong, no one can fully reconstruct why they made a particular decision.

⁵ Karpathy's "march of nines" framing of production reliability.

https://open.substack.com/pub/dwarkesh/p/andrei-karpathy?utm_campaign=post-expanded-share&utm_medium=web

⁶ Gartner (May 2026): >40% of agentic AI projects predicted to be canceled by end of 2027.

<https://www.gartner.com/en/newsroom/press-releases/2026-05-26-gartner-says-applying-uniform-governance-across-ai-agents-will-lead-to-enterprise-ai-agent-failure>

⁷ “Towards a Science of AI Agent Reliability” (arXiv:2602.16666) extensive research into agent reliability.

⁸ [Galileo Agent Reliability Platform](#).

Would you let that intern deploy directly into production, move money, or act inside a regulated system without oversight? Of course not. You would capitalize on their raw intelligence by providing supervision, defining their authority, reviewing their work and gradually increasing their responsibility as their performance, and your trust in them, grows.

Agent Reliability Engineering applies the same principles to intelligent software. The problem is not intelligence. The problem is reliability.

From Intelligence to Adoption



Every investment in AI ultimately succeeds or fails at one of these transitions. Every agent shipped on the allure of raw intelligence and capability alone, skipping reliability and therefore trust, begins life in debt, Reliability Debt.

OpenClaw took the world by storm because it combined intelligence (models) with the capability to act, making it a hobbyist dream come true. However, its initial weakness in reliability, in both security and governance, undermined early enterprise trust and adoption.⁹

Enterprises don't buy raw intelligence, they buy predictable business outcomes.

Every generation of AI has focused on increasing intelligence. The next generation must focus on increasing reliability. Without it, capability never converts into sustained trust or adoption.

⁹ arXiv papers (May 2026): [“Security of OpenClaw Agents...”](#) and [“Security, Privacy, and Ethical Risks in OpenClaw”](#) systematically cover enlarged attack surfaces, skill poisoning, cognitive manipulation, supply-chain issues, and barriers to trustworthy/enterprise adoption.

Agent Reliability Engineering (ARE)

Agent Reliability Engineering (ARE) is the engineering discipline responsible for making autonomous systems predictable, governable, recoverable, and accountable in production. ARE achieves this through explicit identity, authority, contracts, policy enforcement, evaluation, runtime controls, observability, governance, and continuous improvement. Just as SRE transformed operations into engineering, ARE transforms trust in autonomous systems into an engineering problem that can be measured, designed, and continuously improved.

ARE is not another framework, runtime, security product, or evaluation tool. It is the discipline that makes an agent's authority verifiable, its actions observable, and its failures recoverable, before it ever touches production.

It follows the same logic that made Site Reliability Engineering effective. SRE did not eliminate failure; it made failure observable, bounded, and recoverable. ARE does the same for agents that can act. It accepts that probabilistic systems will vary and that errors can compound across steps, tools, and learning loops. The discipline exists to keep those variations within acceptable bounds, to make failures visible, and to ensure the system can recover without losing control or visibility.

Research already shows the divergence: capability scores have risen far faster than consistency, robustness, predictability, and safety.¹⁰ Industry investments in Forward Deployed Engineering, policy-enforcing sandboxes, and governed execution confirm that the problem is now recognized at scale.¹¹

Reliability is an architectural property, not a runtime afterthought. Accountability cannot be retrofitted cleanly after the first incident.

¹⁰ Academic work including “[Towards a Science of AI Agent Reliability](#)” (arXiv:2602.16666) and related studies shows that capability scores have risen far faster than consistency, robustness, predictability, and safety; capability gains have only yielded small improvements in reliability.

¹¹ . Industry responses include [AWS's \\$1B](#) Forward Deployed Engineering investment focused on hands-on deployment and customer self-sufficiency, [Google Cloud's \\$750M](#) partner fund for agentic deployments, [NVIDIA OpenShell's](#) out-of-process policy enforcement, and [Databricks'](#) governed execution and continuous evaluation capabilities.

NORTH STAR

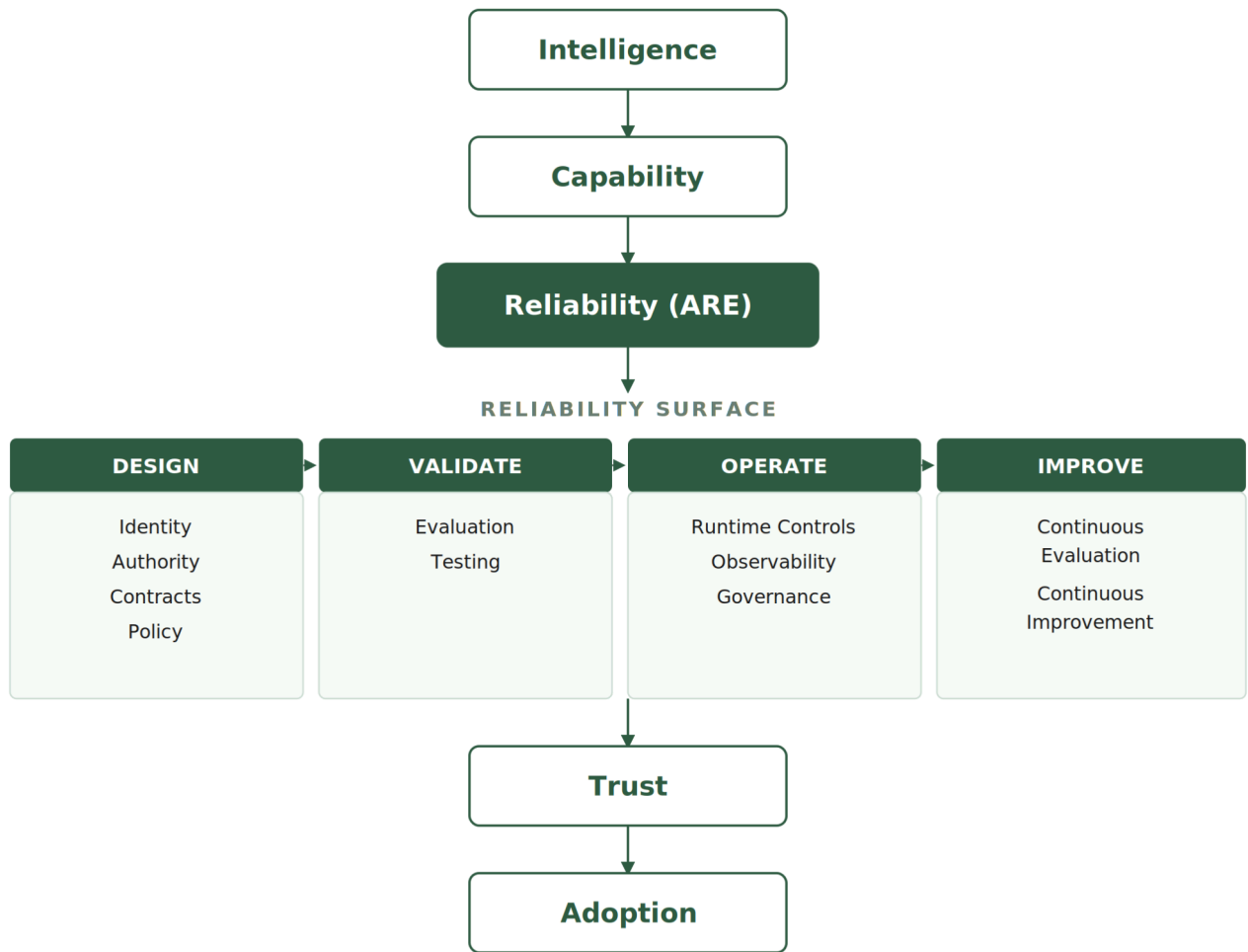
Without reliability, you only have liability.

Embrace reliability. Earn trust. Enable adoption.

The Agent Reliability Engineering (ARE) Surface

Agent Reliability Engineering spans the complete lifecycle required to transform autonomous capability into trustworthy production systems. It is not a single product, framework, runtime, or policy engine. It is the combined effect of every control that determines whether an autonomous system behaves as intended. We call this the Reliability Surface. It includes: identity, authority, contracts, policy, evaluation, testing, runtime controls, observability, governance, continuous evaluation, continuous improvement. These capabilities work together to transform intelligence into reliability, which then builds trust.

Academic research measures reliability through dimensions such as consistency, robustness, predictability, and safety. ARE defines the engineering capabilities required to continuously improve those dimensions.



The Reliability Surface spans the entire engineering lifecycle, from design through continuous improvement, transforming autonomous capability into trustworthy production systems.

Reliability Debt

Reliability Debt is the accumulated risk created when an autonomous system's capabilities grow faster than its reliability engineering. Like Technical Debt, it represents deferred engineering work. Unlike Technical Debt, its primary consequence is not slower feature delivery, it is unpredictable autonomous behavior in production.

Reliability Debt accumulates whenever an agent is permitted to act without sufficient contracts, enforceable policy, evaluation coverage, observability, recovery mechanisms, or runtime controls. As autonomy increases, the debt compounds. Left unaddressed, it becomes the primary barrier to safely

scaling agentic systems. Reliability Debt is the cause behind the North Star phrase: *“Without reliability, you only have liability.”*

Why it matters

Every new capability expands the Reliability Surface:

- a new tool
- a new model
- a new workflow
- a new sub-agent
- a new integration

Organizations routinely invest in expanding what agents *can* do, expanding the Reliability Surface, while failing to invest in Reliability Engineering, thereby creating Reliability Debt. Just as Technical Debt eventually slows software delivery, Reliability Debt eventually limits the safe expansion of autonomy.

Sources of Reliability Debt

Common sources include:

- Missing or ambiguous contracts
- Implicit rather than explicit authority
- Policy that lives only in prompts
- Insufficient evaluation and adversarial testing
- Incomplete decision traces and limited observability
- Absent runtime controls (budgets, circuit breakers, kill switches)
- Unmanaged state and missing recovery paths
- Static governance that never tightens with experience

Symptoms

Reliability Debt usually surfaces before catastrophic failure:

- rising policy violations and authority exceptions
- increasing manual intervention
- growing incident volume and investigation time
- expanding prompt complexity
- difficulty reproducing or explaining decisions
- drift between expected and observed behavior

These are early indicators that reliability engineering has fallen behind capability growth.

Reliability Debt Interest

Reliability Debt Interest is the ongoing operational cost incurred because the debt has not been paid down. Examples include:

- more human approvals and slower deployments
- higher cloud and API spend from unconstrained retries
- longer incident investigations
- frequent policy exceptions
- customer-facing failures
- declining organizational confidence in autonomous operation

Technical Debt costs engineering time. *Reliability Debt costs organizational trust.*

Relationship to the Reliability Maturity Model

Reliability Debt decreases as organizations move up the maturity model:

- **Level 1 – Unmanaged:** Debt grows rapidly and is largely invisible.
- **Level 2 – Observed:** Debt is recognized but still largely unmanaged.
- **Level 3 – Governed:** Contracts, external policy, and decision trails begin to contain it.
- **Level 4 – Accountable:** Debt is measured, authority is progressive, and spawn-chain lineage is enforced.
- **Level 5 – Adaptive:** Continuous evaluation automatically identifies and reduces debt; the system tightens itself.

Organizations do not eliminate Reliability Debt. They continuously manage it as capabilities evolve.

Engineering Principle

Every new autonomous capability introduces potential Reliability Debt.

The purpose of Agent Reliability Engineering is not to slow innovation; it is to ensure that reliability engineering grows at least as quickly as capability. Organizations that continuously reduce Reliability Debt can safely expand autonomy. Organizations that ignore it eventually discover that their greatest limitation is no longer model intelligence, it is operational reliability.

The Meaning and Controls Behind the Words

Each capability answers an engineering question and a business question. The engineering question is what your team asks. The business question is what your leadership, customers, and regulators will ask. The controls are the engineering tools that deliver reliability.

CAPABILITY	ENGINEERING QUESTION	BUSINESS QUESTION	TYPICAL ENGINEERING CONTROLS
Identity	<i>Who is this agent?</i>	Can we identify who acted?	SPIFFE identities, workload identity, signed credentials
Authority	<i>What may it do?</i>	Can it exceed permissions?	Capability tokens, RBAC, least privilege
Contracts	<i>What behavior is expected?</i>	Will it behave consistently?	Schema validation, interface contracts, MCP tool contracts
Policy	<i>What rules are enforced?</i>	Can we prove compliance?	Cedar, OPA, gateway enforcement
Evaluation	<i>Does it actually work?</i>	Can we trust it?	Benchmarks, regression suites
Testing	<i>Does it remain reliable under stress?</i>	Operational risk?	Failure injection, fuzzing, simulation
Runtime Controls	<i>How do we prevent cascading failures?</i>	Blast radius?	Circuit breakers, budgets, kill switches
Observability	<i>Can we reconstruct every decision?</i>	Explainability?	Tracing, replay, immutable logs
Governance	<i>Are all controls working together?</i>	Organizational trust?	Automated policy validation
Continuous Evaluation & Improvement	<i>How do we detect drift and improve over time?</i>	Long-term safety?	Drift detection, retraining triggers

The Principles of Agent Reliability Engineering

These principles define the discipline. They turn reliability from an afterthought into an architectural property of systems that must operate across fluid stacks, evolving knowledge, and short-lived agents.

1. Governance is architectural, not operational.

Identity, authority, and proof must exist before the first action. Accountability cannot be retrofitted after the first incident. Governance that begins only at runtime has already failed.

2. Authority must be explicit, verifiable, and earned.

Every autonomous action must be traceable to explicit authority. Agents may delegate tasks and authority, but accountability must remain traceable through the chain of execution.

3. Agents earn their operational authority through progressive autonomy.

Agents earn greater operational authority only as their measured reliability surface improves, and automatically lose authority when reliability drifts.

4. Constraints must be independently enforceable.

An agent cannot be trusted to enforce its own limits. Constraints must be enforced independently so compromise, prompt injection, or behavioral drift cannot silently remove them.

5. Agents should never rely on implicit assumptions.

Reliable systems make expectations explicit. Contracts define inputs, outputs, authority, constraints, side effects, dependencies, and expected behavior. Ambiguity is the enemy of reliability.

6. Governance must operate at agent speed.

Ephemeral agents and sub-agents that may live only milliseconds make purely hand-coded policy impossible to scale. Agents may be created, delegated, and retired faster than humans can review or approve them. Governance must therefore execute automatically, producing and enforcing controls at the pace autonomous systems operate.

7. Behavior must be governed consistently across the stack.

Contracts express expected behavior, while policies enforce it across models, tools, sandboxes, networks, and infrastructure. These layers must remain aligned so the system behaves consistently from end to end.

8. Reliability is engineered, not observed.

Reliability cannot be inferred from successful demonstrations. It is engineered through explicit controls, continuous measurement, bounded failure, and systematic improvement. Reliability is both a measurable property and an engineering discipline.

9. Every consequential action must be observable and reconstructible.

Every consequential decision must leave a complete, immutable trail sufficient to explain what happened, why it happened, and under whose authority it occurred.

10. Failure must be contained, recoverable, and improvable by design.

Circuit breakers, budgets, halt semantics, and progressive autonomy keep probabilistic error from compounding. Autonomous systems must minimize blast radius, recover predictably, and continuously reduce the likelihood of repeated failure.

11. Reliability begins before deployment.

Reliability is not established by a successful deployment or passing an evaluation. It is engineered from the earliest stages of development and runtime observability strengthens it but does not replace it. The most reliable autonomous systems are engineered for reliability long before they execute their first action.

Reliability Maturity Model

Modeled loosely on the leveled, per-capability structure of frameworks like SLSA in software supply chain security, this maturity model lets organizations assess reliability incrementally rather than as a pass/fail gate.

Level 1 — Unmanaged

Agents run with human-equivalent or shared credentials. No distinct identity. The agent enforces its own constraints via prompt instructions only, nothing external checks it. Logging is incidental, not designed for reconstruction. No containment: a failure has no defined blast radius.

SIGNALS

- You cannot reliably answer “which specific agent took this action?” after the fact
- There is no independent mechanism that can stop an agent once it has started acting

Level 2 — Observed

Agents have distinct identities, but no verified authority lineage; you know which agent acted, not what it was actually allowed to do. Policy exists as a document, not an enforced boundary. Basic action logs exist but aren't immutable or complete enough to reconstruct a decision. A human can pull the plug manually; there's no automatic circuit breaker.

SIGNALS

- You can identify the agent, but you cannot prove what it was actually authorized to do
- Stopping a misbehaving agent still requires a human to notice and intervene

Privacy Note: Complete, immutable decision trails create an inherent tension with privacy regulations such as GDPR's right to erasure. ARE does not require perpetual, unredacted retention of every prompt, retrieval, or intermediate state. Production systems must pair observability with retention policies, selective redaction, and purpose limitation so that reconstruction remains possible for accountability while personal data can still be deleted or minimized when required by law. The goal is auditable reliability, not permanent surveillance.

Level 3 — Governed

Every agent has a verifiable identity, and authority is explicitly scoped per task. Policy enforcement lives outside the agent (gateway, sandbox, or proxy, not the model itself). Full decision trails exist: prompts, tool calls, responses, retrievals, timestamped and preserved. Circuit breakers and budgets are defined per agent class, and failures halt automatically rather than propagating. Reliability metrics are tracked and reviewed on a cadence.

SIGNALS

- An external system (not the model) can independently block or halt agent actions
- You can reconstruct most consequential decisions well enough to explain them to a customer or auditor

Level 4 — Accountable

Sub-agents inherit and can prove their authority lineage back through the spawn chain — not just “an agent did this” but “this specific agent, spawned by this parent, with this scope, did this.” Policy is generated dynamically from contracts and templates so short-lived sub-agents don't fall outside coverage. Decision trails meet regulatory-grade explainability. Autonomy is progressive; agents earn expanded authority through demonstrated reliability and lose it automatically on drift.

SIGNALS

- You can prove the full authority chain for any sub-agent action back to the original task
- Agents automatically gain or lose scope based on measured reliability, not just static configuration

Level 5 — Adaptive

Full spawn-chain authority verification holds even for sub-agents that live milliseconds. Policy generation keeps pace with agent evolution in real time — no human is the bottleneck. The reliability surface feeds back into policy generation automatically, so the system tightens itself after incidents rather than waiting for a human postmortem. Critically, the tooling is usable by non-specialists, so reliability isn't gated behind a small team who understands the internals.

SIGNALS

- New or short-lived agents receive appropriate policy without manual rule-writing
- Reliability problems detected in production automatically tighten future controls
- People who are not reliability specialists can still understand and adjust the system's behavior

ARE is not about slowing agents down. It is about making them trustworthy enough to move fast.

The ARE Toolbox

Category	Examples
Failure Containment	Circuit breakers, kill switches, execution budgets
Recovery	Retries, rollback, compensation, replay
Trust	Identity, capability tokens, least privilege
Validation	Contracts, schema validation, state validation
Verification	Continuous evaluation, regression testing, chaos engineering
Operations	Observability, tracing, policy enforcement, drift detection

Call to Action

Treat governance as a design primitive. Identity, authority, contracts, and reconstructible trails must be present from the first action. Adopt shared tenets, contribute patterns, and insist that both the policy governing the agent and the infrastructure policies governing its stack are explicit, enforceable, and reviewable.

For leaders: fund the reliability surface the same way you fund capability. Model intelligence alone does not produce trustworthy agents. Budget for measurement, policy generation, observability, containment, and continuous evaluation. Reliability is not a cost center that slows delivery; it is what makes sustained speed possible.

This discipline does not belong to one company. It belongs to everyone building autonomous systems. Agent Reliability Engineering will only succeed if it is developed in the open, with shared vocabulary, portable approaches, and common reliability surfaces emerging through collective practice. This discipline, like this document, will evolve with learning and feedback.

The Next Discipline

Site Reliability Engineering transformed distributed computing by making complex systems observable, recoverable, and dependable. Agent Reliability Engineering extends that philosophy to autonomous systems.

Intelligence alone is insufficient. Trust must be engineered. Accountability must be architectural. Authority must be earned. Reliability must become continuous.

As autonomous software becomes foundational to every industry, reliability will become as indispensable to agents as SRE became to cloud infrastructure.

This is not the end of software engineering.

It is the beginning of its next discipline.

*The future will belong not simply to intelligent agents —
but to reliable ones.*